

AI Search Lab 1 – Conversation Summary

1. Core Search Concepts

State

A state represents the current situation of the agent.

Example:

- `(1,1)` in a grid means the agent is currently at row 1, column 1.

Question answered:

"Where am I right now?"

Node

A node contains more information than a state.

A node stores:

- State
- Parent node
- Action used to reach it
- Path cost
- Depth

Question answered:

"Where am I, and how did I get here?"

Example:

`Node( state=(1,1), parent=previous_node, action="UP", path_cost=3 )`

Frontier

The frontier is the collection of nodes that have been discovered but not yet explored.

Think of it as a waiting list.

Example:

Frontier: [A, B, C]

The search algorithm repeatedly removes a node from the frontier and explores it.

Expansion

Expanding a node means:

1. Look at the node's state.
2. Find all legal actions.
3. Apply each action.
4. Generate child nodes.

Example:

State: (1,1)

Actions: UP, DOWN, LEFT, RIGHT

Children: (0,1) (2,1) (1,0) (1,2)

Expansion generates new nodes that may later enter the frontier.

2. Grid Problem Functions

is_goal(state)

Checks whether a state is the goal.

Example:

```
return state == goal
```

actions(state)

Returns all valid actions from a state.

Example:

```
["UP", "DOWN", "LEFT", "RIGHT"]
```

result(state, action)

Returns the next state after performing an action.

Example:

(1,1) + UP → (0,1)

action_cost(state, action, next_state)

Returns the cost of a move.

Usually:

return 1

3. Understanding expand()

Implementation:

```
s = node.state

for action in problem.actions(s):
    s_prime = problem.result(s, action)

    cost = (
        node.path_cost
        + problem.action_cost(s, action, s_prime)
    )

    yield Node(
        state=s_prime,
        parent=node,
        action=action,
        path_cost=cost
    )
```

Logic:

- Take current state.
- Generate all legal actions.
- Compute resulting states.
- Create child nodes.
- Yield child nodes one at a time.

Purpose:

Turn one node into all reachable child nodes.

4. Breadth-First Search (BFS)

Main Idea

Explore nodes level by level.

Uses:

- Queue
- FIFO (First In First Out)

Example order:

A B C D E

Frontier Structure

```
frontier = deque([node])
```

Remove nodes using:

```
node = frontier.popleft()
```

Reached Set

```
reached = {initial_state}
```

Used to avoid revisiting states.

Important insight:

Because states are added to `reached` immediately when discovered, a separate frontier-membership check is unnecessary.

Metrics

nodes_expanded

Counts how many nodes were explored.

max_frontier_size

Tracks the largest frontier size seen during search.

Implementation:

```
max_frontier_size = max(
    max_frontier_size,
    len(frontier)
)
```

Purpose: Measure memory usage.

5. Depth-First Search (DFS)

Main Idea

Explore one branch as deeply as possible before backtracking.

Uses:

- Stack
 - LIFO (Last In First Out)
-

Frontier Structure

```
frontier = [node]
```

Remove nodes using:

```
node = frontier.pop()
```

Difference from BFS

BFS:

```
popleft()
```

DFS:

```
pop()
```

Everything else is largely the same.

6. Depth-Limited Search (DLS)

Main Idea

DFS with a maximum depth limit.

Possible Outcomes

Success

Goal found.

Cutoff

Search reached the depth limit.

This does NOT mean failure.

It means:

"I stopped because the depth limit prevented further exploration."

Failure

No solution found and no cutoff occurred.

Important Difference from BFS/DFS

Do NOT use a global reached set.

Instead use path-cycle checking:

```
state_is_on_path(node, child.state)
```

Purpose:

Avoid cycles only along the current path.

Recursive Structure

1. Goal test.
2. Depth limit test.
3. Expand children.
4. Recurse on each child.
5. Track whether any cutoff occurred.

6. Return:

7. success

8. cutoff

9. failure

Parameters Explained

problem

Defines the search world.

node

Current position in the search tree.

limit

Maximum depth allowed.

metrics

Stores:

- nodes_expanded
- max_stack_size

current_stack_size

Tracks recursion depth for memory statistics.

7. Iterative Deepening Search (IDS)

Main Idea

Run DLS repeatedly with increasing depth limits.

Example:

limit = 0 limit = 1 limit = 2 limit = 3 ...

until success.

Why IDS Exists

Combines:

- BFS completeness
 - BFS optimality (for unit costs)
 - DFS memory efficiency
-

IDS Logic

For each depth limit:

```
result = dls.search(problem, limit)
```

If:

- success → return success
 - failure → stop and return failure
 - cutoff → try a larger limit
-

Metrics

Accumulate:

```
nodes_expanded += result.nodes_expanded
```

Track:

```
max_stack_size
```

across all DLS runs.

Key Mental Models Learned

State:

Where am I?

Node:

Where am I and how did I get here?

Frontier:

States discovered but not yet explored.

Expand:

Generate children from a node.

BFS:

Explore wide.

DFS:

Explore deep.

DLS:

Explore deep, but stop at a depth limit.

IDS:

Repeatedly run DLS with larger limits until the goal is found.